

Protocol

Protocol for automated analysis of biological images using Python code

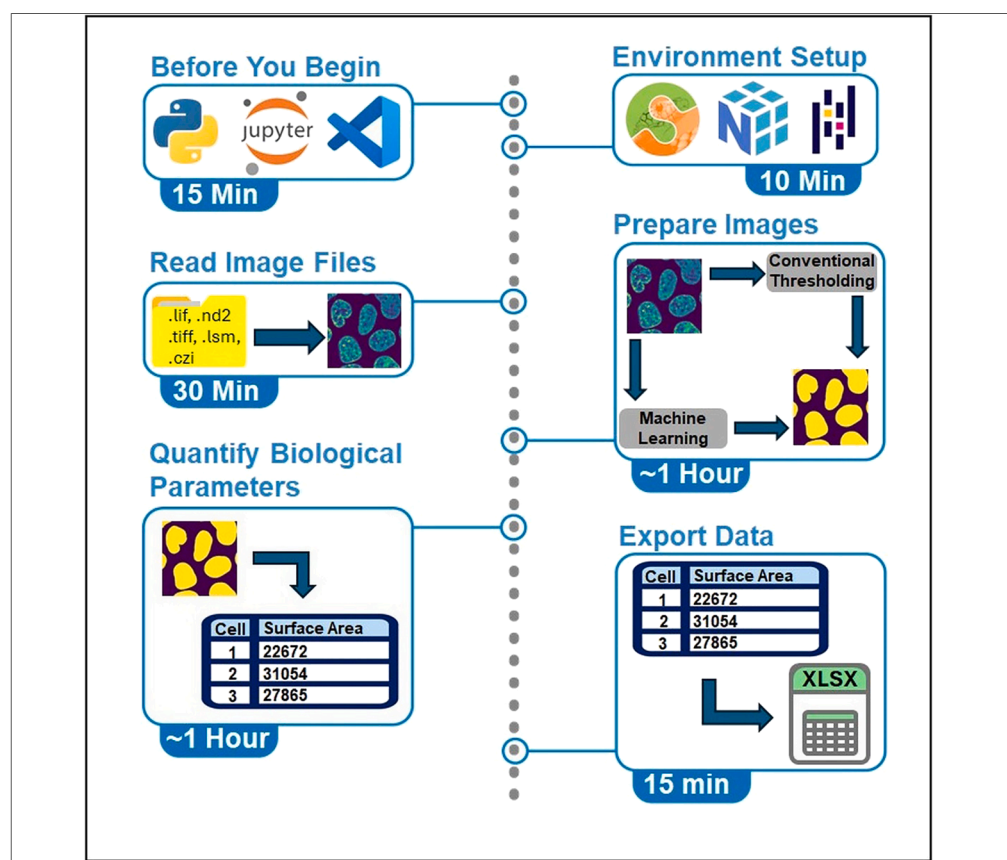


Image analysis in Python provides greater flexibility and an open-source alternative to commercially available software. In this protocol, we present steps for quantifying features in various common biological images in a manner that is tailored to scientists with little coding background. Using example experiments, we outline steps to read in a variety of image file types and prepare them for quantitative analysis. We then discuss approaches for segmenting images, extracting measurements from segmented images, and exporting data.

Publisher's note: Undertaking any experimental protocol requires adherence to local institutional guidelines for laboratory safety and ethics.

Benjamin Sacks, Jiya Mody, Yiming Huang, ..., Alejandro Mosera, Alessandro Crimi, Eleanna Kara
ek851@rwjms.rutgers.edu

Highlights
Steps for the analysis of confocal images through Python

Guidance on the usage of machine learning for cell segmentation

Troubleshooting of common problems encountered during imaging analysis by Python

Sacks et al., STAR Protocols 7, 104368
March 20, 2026 © 2026 The Author(s). Published by Elsevier Inc.
<https://doi.org/10.1016/j.xpro.2026.104368>



Protocol

Protocol for automated analysis of biological images using Python code

Benjamin Sacks,^{1,3,5,6} Jiya Mody,^{1,3} Yiming Huang,^{1,4} Sunghoo Jung,^{1,4} Robert Kimelman,^{1,4} Elena Eubanks,¹ Elyse Guadagno,¹ Jose Cevallos,¹ Ananya Guntur,¹ Neeharika Rao Suvvari,¹ Aryan Doshi,¹ Nitya Ravinutala,¹ Alejandro Mosera,¹ Alessandro Crimi,² and Eleanna Kara^{1,6,7,*}

¹Department of Neurology, Robert Wood Johnson Medical School, Institute for Neurological Therapeutics at Rutgers, Rutgers Biomedical and Health Sciences, Piscataway, NJ 08854, USA

²AGH University of Krakow, al. Adama Mickiewicza 30, 30-059 Kraków, Poland

³These authors contributed equally

⁴These authors contributed equally

⁵Present address: Department of Radiology, Athinoula A. Martinos Center for Biomedical Imaging, Massachusetts General Hospital, Charlestown, MA 02129, USA

⁶Technical contact

⁷Lead contact

*Correspondence: ek851@rwjms.rutgers.edu
<https://doi.org/10.1016/j.xpro.2026.104368>

SUMMARY

Image analysis in Python provides greater flexibility and an open-source alternative to commercially available software. In this protocol, we present steps for quantifying features in various common biological images in a manner that is tailored to scientists with little coding background. Using example experiments, we outline steps to read in a variety of image file types and prepare them for quantitative analysis. We then discuss approaches for segmenting images, extracting measurements from segmented images, and exporting data. For complete details on the use and execution of this protocol, please refer to Eubanks et al.¹

BEFORE YOU BEGIN

The advent of high content microscopy has resulted in the generation of large imaging datasets that require standardized and automated analyses. This is typically challenging to achieve through commercially available software or web-based browsers. Computer vision in the Python programming language provides automation and flexibility with microscopic image quantification for a variety of experiments examining subcellular structures and cellular functions. Additionally, installing and getting started with Python is more user-friendly than other programming languages, which is further facilitated by several pre-existing libraries. Python and code editors (e.g., Visual Studio Code) can both be downloaded for free, and the Jupyter Notebook extension and file type integrate code and output into sequential fragments.²

Image analysis requires an understanding of fundamental aspects of Python programming, including strings, conditionals, and iteration (i.e., loops). Collections of characters are stored as **strings**, denoted with quotation marks and used to represent file paths or image information. **Conditionals**, syntactically represented with *if*, allow certain commands to execute only when a particular requirement is met. Lastly, **iteration** allows a set of commands to be run multiple times without repeating the code written. Iteration falls into two categories: *while* loops, which repeatedly execute until a provided conditional is no longer met, and *for* loops, which run commands for each piece of



data in a collection. Iteration allows many images to be run through the same analysis with little additional effort.

While various methods for preprocessing and measuring images are described, analysis procedures in this protocol follow a general structure of reading, preprocessing, measuring, and reporting. When processing an image, the file must first be read into the Python environment. While file types will vary based on the microscope used, many Python libraries exist for reading image files.^{3–6} Read files are represented as multidimensional arrays, which can be prepared using numpy.⁷ Two-dimensional arrays can then be preprocessed by applying filters (e.g., Gaussian blur), standardization (i.e., contrast adjustment), and finally thresholding to separate the foreground and background of the image.⁸ This generates a segmented image that can be measured and reported. The following describes the pipeline of image analysis using Python, highlighting options for image preprocessing and measurement of cell function and morphology. While the general pipeline remains the same, there is flexibility in preprocessing and measurement when using Python for image analysis that offers greater versatility when compared to other software. While much of the protocol is written for a variety of applications, some optional sections outline specific methods used for the analysis of intracellular structures, such as proteinaceous inclusions with fluorescent labeling in cultured cells.

This protocol represents a step-by-step guide for the analysis of confocal microscopy images using real life examples published in our recent manuscript¹ and builds upon existing protocols and resources.⁹ In particular, it focuses on the most commonly used analyses, which can be challenging to accurately implement through ImageJ or other software: z-stack volumetric quantifications, mean fluorescence intensity quantifications, colocalization analysis, object identification and quantification of size and shape. Additionally, this protocol gradually scales from basic segmentation techniques to more modern methods for segmentation, introducing readers to a variety of segmentation methods and providing a general “crash course” on various approaches. Original code and data used in the protocol is available at <https://doi.org/10.5281/zenodo.17757130> as well as https://github.com/eleanakara/Kara-Lab/tree/main/image_analysis_methods_2025.

Innovation

This protocol outlines the complete process for reading, preprocessing, and measuring features of microscopic biological images. This process requires an understanding of computer programming in Python, which we introduce to the user in a manner that is friendly for those who are new to programming.

This includes an introduction to machine learning based segmentation tools. Several pre-trained image segmentation models are listed and organized alongside code for setup and basic use to facilitate comparison between multiple options. A list of publicly available image databases is also included for reference and model testing. As an example, a custom UNet model designed for the segmentation of microscopic images at a cellular level is also detailed both for use and as a foundation for further customization, if required.

After detailing how to segment images, we provide examples of feature extraction and measurement of segmented images used in basic biological research. These examples come from the associated publication to further contextualize how image analysis can be used to answer real research questions.

Preparation #1: Install Python and software for editing Jupyter Notebooks

⌚ Timing: 15 min

1. Install Python 3 from <https://www.python.org/downloads/>.

2. Check that Python 3 is installed by running the following command in your terminal.
[Troubleshooting 1](#).

```
>python -version
```

Note: The terminal can be located by entering 'cmd' in the system search bar. On Windows, the system search bar is found by pressing the Windows key. On Mac, the system search bar is found by pressing command and the space bar. Different operating systems will require a different command to call Python.

3. Install a code editor capable of editing Jupyter Notebooks. Visual Studio Code is one option. Steps for installing Visual Studio Code are shown below.
 - a. Download and install Visual Studio Code from <https://code.visualstudio.com/download>. Select the correct download for your operating system.
 - b. Once installed, navigate to the "Extensions" tab located as an icon on the right to install extensions for Python and Jupyter.
 - i. Install the extension for Python support by entering "Python" into the search bar and selecting the top result.
 - ii. Install the extension for Jupyter Notebook support by entering "Jupyter" into the search bar and selecting the top result.
 - c. To verify that Visual Studio Code has Jupyter Notebook support select "File", then "New File". An option to create a new Jupyter Notebook should appear in the drop-down menu.

Preparation #2: Install Conda for virtual environment management

⌚ Timing: 15 min

4. Install Conda from <https://www.anaconda.com/docs/getting-started/miniconda/install#quickstart-install-instructions>.

⚠ **CRITICAL:** Miniconda installation differs between Windows, MacOS, and Linux systems. The proper commands for installation should be copied and run in the terminal (see preparation #1). On windows, the terminal is also referred to as the command prompt.

5. Activate Conda by running the activation script from the terminal.
 - a. On Windows.

```
>.\miniconda3\Scripts\activate.bat
```

- b. On MacOS and Linux:
 - i. Run the following command to refresh the terminal.

```
> source ~/miniconda3/bin/activate
```

- ii. Initialize conda on every open shell.

```
> conda init -all
```

6. In the terminal, verify that Miniconda has been properly installed.

```
>conda --version
```


Note: If Conda is not properly installed or activated, running any command that calls Conda will return an error stating that Conda is not found or recognized.

⚠ **CRITICAL:** When opening a new terminal window, the command in step two must be run again to activate Conda in the new window.

KEY RESOURCES TABLE

REAGENT or RESOURCE	SOURCE	IDENTIFIER
Software and algorithms		
Python v >= 3.12.1	Python Software Foundation	https://www.python.org/downloads/source/
Visual Studio Code v >= 1.95 (or other IDE able to edit Jupyter Notebooks)	Microsoft	https://code.visualstudio.com/
Scikit-image (skimage) v >= 0.24.0	Walt et al. ⁸	https://scikit-image.org/
Numpy v >= 2.1.3	Harris et al. ⁷	https://numpy.org/
Pandas v >= 2.2.3	McKinney & others ¹⁰	https://pandas.pydata.org/
Matplotlib v >= 3.9.2	Hunter ¹¹	https://matplotlib.org/
Pytorch v >= 2.8.0	Paszke et al. ¹²	https://pytorch.org/
Torchvision	Maintainers & Contributors ¹³	https://pytorch.org/project/torchvision/
Cellpose	Stringer et al. ¹⁴	https://pytorch.org/project/cellpose/
Pillow	Clark ¹⁵	https://pytorch.org/project/pillow/
Readlif	Negretti ⁵	https://pytorch.org/project/readlif/
Nd2	Lambert ⁴	https://pytorch.org/project/nd2/
Tiffle	Gohlke ³	https://pytorch.org/project/tiffle/
Aicspylibcz	Watkins ⁶	https://pytorch.org/project/aicspylibcz/
Czfile	Gohlke ¹⁶	https://pytorch.org/project/czfile/
Deposited data		
Raw data	Eubanks et al. ¹	https://zenodo.org/records/17757130

STEP-BY-STEP METHOD DETAILS

Environment setup

⌚ **Timing:** 10 min

This step details how to create a virtual environment using Conda and install the Python libraries required for this procedure.

1. Initialize a virtual environment.

Note: Virtual environments are specific directories used to install and manage modules separate from other projects (optional step). We recommend using a virtual environment, as they keep dependencies for separate projects isolated and prevent conflicts (e.g., in one project one library may depend on an older version of another library compared to another project).

a. In the terminal, create a virtual environment with the Conda using the following command.

```
>conda create --name my_env_name
```

b. Activate the virtual environment.

Note: Any modules installed while the virtual environment is active will only be accessible from the virtual environment (unless installed globally as well).

```
>conda activate my_env_name
```

- c. To enable Jupyter Notebook compatibility, install the package 'ipykernel' and create a Jupyter Notebook kernel associated with the virtual environment.

```
# install the package ipykernel
>pip install ipykernel
# initialize a Jupyter Notebook Kernel from the virtual environment
>python -m ipykernel install --user --name=new_virtual_env --Display-name="Name to display"
```

Note: When installing the Jupyter Notebook kernel, the parameter 'name' must match the given name of the virtual environment, while the parameter 'Display-name' can be anything, but should be descriptive and recognizable.

- d. To verify that the virtual environment is now compatible with Jupyter Notebooks, open Visual Studio Code (or other integrated development environment) and open or create a new Jupyter Notebook file. If the kernel has been successfully installed, it will be an available option. In Visual Studio the kernel can be selected after clicking 'Select Kernel' in the top right corner of the screen. Then select 'Jupyter Kernel' and the display name of the newly installed kernel should be visible.
2. Install required libraries. This can be accomplished by calling Python within the terminal and entering the following commands. If you wish to use a virtual environment, verify that the environment has been activated. If not, activate it prior to installing libraries (see step 1).

```
>pip install numpy
>pip install scikit-image
>pip install matplotlib
>pip install pandas
```

Note: Pip is not required for the installation of Python libraries, as other software (e.g., Conda) can perform a similar function. Package installation will be shown with pip; however, any package manager may be used.

3. Import required libraries prior to image processing.

```
# can import full libraries
>import numpy as np
>import matplotlib.pyplot as plt
>import pandas as pd
>import skimage
# alternatively, individual functions can be imported from libraries
>from skimage.filters import threshold_otsu
```

Note: If scikit-image is loaded in full, then each function must be called from its module. For example, the function “threshold_otsu()” must be called from filters module using “skimage.filters.threshold_otsu()”. Modules are python files that store one or more functions, allowing them to be easily shared and run across machines. Importing a module loads the functions it contains and allows them to be run on the current machine.

Reading image files into Python

⌚ Timing: 30 min

This step provides instructions for reading in .lif, .nd2, .lsm, .tiff and .czi files. While this process varies slightly depending on the file type, it always transforms the image data into a multidimensional numpy array. The array will contain 3 or more dimensions depending on the number of slices or mosaic the image contains. Additionally, we outline additional steps for parsing graphical annotations from the metadata of .czi files.

4. Based on the image file types, follow the specific directions to read the image file into Python. Below are instructions for reading .lif, .nd2, .lsm, .tiff, and .czi file types.
 - a. Reading .lif (Leica proprietary) image files.
 - i. Install the library readlif for reading .lif image files and import the library.⁵

```
>!pip install readlif
>import readlif
```

Note: Adding an exclamation point to the beginning of commands allows the Jupyter Notebook to execute them as if they were called inside the terminal. Modules can be installed directly from the terminal with the same commands after removing the exclamation point.

Note: Examples of .lif files are available in many public repositories, including the one found here: <https://www.ebi.ac.uk/biostudies/BiolImages/studies/S-BSST749?query=.lif>.

- ii. Using readlif, read the file in as a LifImage object by supplying a file path. This simultaneously selects a single image in the file if multiple are present.

```
>lif_file = "path/to/your/file/"
>lif_image = readlif.reader.lifFile(lif_file).get_image(2)
```

Note: Python uses zero based indexing (i.e., items in collections are counted starting from zero instead of one). In the above chunk, the third image would be returned from calling index two.

- iii. Select a single frame in the following dimensions: z (z axis), t (time point), c (channel), m (mosaic image). If each dimension is only of size one, keep all parameters equal to zero (as shown). Image preprocessing and measurement work best when applied to a single image at a time, so a single frame must be selected.

```
>lif_frame = lif_image.get_frame(z=0, t=0, c=0, m=0)
```

- iv. Convert the frame from a Pillow image object¹⁵ to a numpy array.

```
>lif_array = np.array(lif_frame)
```

Note: If not previously installed, Pillow is installed as a dependency of readlif and does not need to be installed independently.

- b. Reading .nd2 (Nikon proprietary) image files.
 - i. If not installed, install the library nd2 for reading Nikon image files and import the library.⁴

```
>!pip install nd2
>import nd2
```

- ii. Read the image in as an array and view the shape of the array.

```
>nd2_array = nd2.imread(nd2_file_path)
>print(nd2_array.shape)
```

- c. Reading .lsm (ZEISS proprietary) or .tiff image Files.
 - i. If not installed, install the library tiff file for reading .tiff and similarly structured file types, then import the library.³

```
>!pip install tiff file
>import tiff file
```

- ii. Read the file as an image object.

```
>lsm_tiff_image = tiff file.TiffFile(lsm_tiff_file)
```

- iii. Convert the image object to an array and view the shape.

```
>lsm_tiff_array = lsm_tiff_image.asarray()
>print(lsm_tiff_array.shape)
```

- d. Reading .czi (ZEISS proprietary) image files. [Troubleshooting 2](#).
 - i. If not installed, install and import the function CziFile from aicspylibczi.⁶

```
>!pip install aicspylibczi
>from aicspylibczi import CziFile
```

- ii. Read the .czi file into Python as a numpy array and remove any axes with length equal to one.

```
>czi file = CziFile(image_path)
>image = czi file.read_image(S=1)[0]
# print length of each axis
>print(image.shape)
>image_squeezed = np.squeeze(image, axis = (first_axis, second_axis, etc))
```

Note: For analysis that uses metadata, image files containing metadata (e.g., .czi, .lsm) are preferred over .tiff file formats.

Note: While “numpy.squeeze” may be used to automatically remove axes with length one, it is recommended to manually select axes in order to prevent losing any necessary data. Instead, the length of each axis should be printed and the axes to drop should be specified within the parameter ‘axis’ when squeezing.

5. If working with .czi files that have been manually annotated in ZEN, parse graphic annotations from the file metadata (optional step).

△ CRITICAL: .czi files must be read through a different method to properly extract metadata. Refer to Troubleshooting 2 for details.

- a. Load the required library for parsing an XML tree.

```
>import xml.etree.ElementTree as ET
```

- b. Select metadata from previously read.czi file and convert to an XML tree.

```
>xml_mdata = czi_image.metadata()
>xml_root = ET.fromstring(xml_mdata)
```

- c. Locate the parent node of the graphic annotation data using the node attribute ‘Name’ and loop through each child node to identify whether annotation is a circle or ellipse.

```
# get the first location of graphic annotations and the list of elements
>ann_layer = root.find(".//*[@Name='Annotation']")
>elems = ann_layer.find("Elements")
>ellipse = True # identify if annotation is circle or ellipse
>for child in elems.findall("*"): # loop through all children
    if child.tag == "Circle":
        ellipse = False
        elem = child
    elif child.tag == "Ellipse":
        ellipse = True
        elem = child
```

- d. Based on the shape of the annotation, parse and store the annotation coordinates in a pandas dataframe.

```
>geom_data = []
>elem_geom = elem.find("Geometry") # pull location from element
>count = 0
>for child in elem_geom.findall("*"): # loop through coordinates
    # if circle, grab first three children
    if (ellipse and count < 4):
        geom_data.append(float(child.text))
```

```
elif (count < 3): # if ellipse, grab first four children
    geom_data.append(float(child.text))
    count+=1
>print(geom_data)
```

Image preparation: Level adjustment, thresholding, and segmentation

⌚ Timing: ~1 h

This step outlines the automated correction of image exposure and separation of foreground from background. Ultimately, this creates a mask, a binary array the same size as the original image, that identifies which pixels are associated with the object or objects of interest so they may be measured.

6. Apply a filter to denoise or smooth the image. [Troubleshooting 3](#).
 - a. Apply a Gaussian filter, which averages pixel values using a Gaussian distribution.

```
>filtered_image = skimage.filters.gaussian(image)
```

- b. Alternatively, apply a different filter, including a median filter, which replaces pixels using the median of each grouping.

```
>filtered_image = skimage.filters.median(image)
```

7. Adjust the contrast of the image to better separate the foreground and background of the image using one of the following options.

Note: While steps and details (i.e., parameters) for filtering, adjusting, and thresholding may vary based on the image and the measurement, the steps detailed are used across experiments. An outline of the steps is described in [Figure 1](#).

- a. Adjust image contrast using sigmoid rescaling. A cutoff value can be set for the function to shift the center of the intensity function to the left (if less than 0.5) or right (if greater than 0.5).

```
>adjusted_image = skimage.exposure.adjust_sigmoid(image, cutoff = 0.25)
```

Note: This first rescales pixel values between 0 and 1, then adjusts pixel values along a sigmoid function.¹⁷

- b. Adjust image contrast with histogram equalization, which evens the intensity histogram of the image.

```
>adjusted_image = skimage.exposure.equalize_hist(image)
```

- c. Adjust image contrast with limited adaptive histogram equalization, which provides local contrast adjustment.

```
>adjusted_image = skimage.exposure.equalize_adapthist(image)
```

Note: When initially writing the procedure for image adjustment, test multiple filtering and adjustment methods with multiple images and compare the quality of segmentation. Filtering and adjustment are recommended, but not always necessary.

8. Generate a threshold for separating foreground and background using one of the methods below.

a. Generate a threshold using Otsu's method.

```
>threshold = skimage.filters.threshold_otsu(adjusted_image)
```

Note: Otsu's thresholding algorithm identifies the threshold value that leads to the lowest variance within groups of pixel values both above and below.¹⁸

b. Alternatively, use another common method for generating a threshold, including normalizing pixel values and then manually selecting a threshold.

i. Normalize each pixel's value between 0 and 1.

```
>adjusted_image = (adjusted_image - adjusted_image.min()) / (adjusted_image.max() - adjusted_image.min())
```

ii. Choose a value between 0 and 1 as a threshold.

```
>threshold = 0.15
```

9. Generate a binary mask to separate foreground and background pixels based on the threshold.

[Troubleshooting 6](#).

```
>image_mask = adjusted_image > threshold
```

10. Clean the binary mask by filling in or removing holes, objects of a particular size, or objects touching the border of the image (optional step).

```
# remove objects with an area smaller than the given threshold.
>image_mask_clean = skimage.morphology.remove_small_objects(image_mask, min_size=50)

# fill in holes in the mask with an area smaller than the given threshold.
>image_mask_clean = skimage.morphology.remove_small_holes(image_mask,
area_threshold=50)

# remove any object touching the edge of the image
> image_mask_clean = skimage.segmentation.clear_border(image_mask)
```

△ **CRITICAL:** After each step of processing, visually inspect multiple images to ensure that adjustment and segmentation occur properly. See step 22 for instructions on printing images.

11. Segment individual cells with Cellpose, a machine learning model designed for segmenting cells in biological image analysis (optional step).¹⁴ [Troubleshooting 4](#).

△ **CRITICAL:** Code below is appropriate for Cellpose version 3 and earlier. If working with Cellpose version 4 or later, syntax will differ from what is shown. This includes how the model is called and how channels are supplied. Specific versions of a package can be installed by supplying the version number after the library name. For example, the command 'pip install cellpose==3.0.1' would install cellpose version 3.0.1.

- a. If not already installed, install and import Cellpose.

```
>!pip install cellpose
>from cellpose import models, utils, io
>model = models.Cellpose(gpu=False, model_type='cyto')
```

△ **CRITICAL:** The model must also be initialized prior to use. The model type can be set to 'cyto' if cytoplasm is the main channel for segmentation or 'nuclei' if nuclei will be shown in the main channel.

Note: The parameter 'gpu' allows the user to run Cellpose with the computer's CPU or with associated GPUs. Running Cellpose or other machine learning algorithms will be significantly faster when using GPUs compared to the computer's CPU. To enable the use of GPUs, PyTorch must be installed with CUDA support (see step 12).

- b. Load and apply a Cellpose model to generate cell segmentation masks. [Troubleshooting 4](#).

```
# if using only a green channel is used
>channel_data = preprocessed_green_channel
>channels = [0,0]

# if a green and blue channel are used
>channel_data = np.stack([green_channel, blue_channel], axis=0)
>channels = [2, 3]

# generate cell segmentations
>diameter = 60 # multiple diameter values should be tested
masks, _, _ = model.eval(channel_data, diameter, channels)
```

△ **CRITICAL:** A channel outlining the cytoplasm of cells (referred to here as a green channel) should be used for Cellpose segmentation. The channel should be preprocessed to optimize cell contrast from the background. A channel showing nuclei (referred to here as a blue channel) may also be used in place of or in addition to the green channel. In our experience, including a nuclear channel in addition to a cytoplasmic channel improves the accuracy of the segmentation. The number and type of channels must be specified as a list of two integers using the key: 0=grayscale, 1=red, 2=green, 3=blue, where the second integer specifies an optional second channel. The diameter parameter should be adjusted to best fit the image.

- c. To generate three dimensional segmentation when working with z-stacks (i.e., three dimensional images) run the two dimensional Cellpose segmentation on each slice then enable cross-slice stitching.

```
# if only a green channel is used
>channel_data = np.stack(
    [(preprocessed_green_channel[z]) for z in range(preprocessed_green_channel[0])],
    axis=0)
>channels = [0, 0]
# if a green and blue channel are used
>channel_data = np.stack([green_channel, blue_channel], axis=1)
>channels = [2, 3]
# generate cell segmentations
>diameter = 60 # multiple diameter values should be tested
masks, _, _ = model.eval(channel_data,
    diameter,
    channels,
    channel_axis=1,
    do_3D=False,
    anisotropy=10,
    min_size=10000,
    stitch_threshold=0.5)
```

Note: The parameter 'channel_axis' must be supplied with the dimension of the array that holds channel information (e.g., nuclei, cytoplasm). If segmenting in three dimensions but 'stitch_threshold' is defined as greater than zero, then the parameter 'do_3d' should be defined as false. The parameter 'anisotropy' must be supplied with the ratio of z slice to pixel size (e.g., if z slices are every 100 microns and each pixel is 10 microns wide and tall, set anisotropy to 10). The parameter 'min_size' should be supplied with the threshold (in pixels) below which objects will not be segmented. Finally, the parameter 'stitch_threshold' defines how closely segmented regions in different slices should be attached when forming a volume. 'Stitch_threshold' ranges from 0 to 1, with larger thresholds attaching regions at larger distances.

- d. Visualize Cellpose masks.

```
# visualize every cell mask together
>plt.imshow(masks)
>plt.show()
# visualize a single cell mask
>cell_three_mask = mask == 3
>plt.imshow(cell_three_mask)
>plt.show()
```

△ **CRITICAL:** When first using Cellpose, visualize Cellpose segmentation with various diameters to see which works best with the set of images.

Note: Each individual cell mask will have its own label. Each label is assigned a unique integer counting from 1 and can be visualized independently of other masks using a conditional (e.g., `cell_three_mask = masks == 3`).

Image segmentation using advanced machine learning analysis

⌚ **Timing:** ~15 h (highly dataset dependent)

This step provides an optional and more advanced option for image segmentation, detailing the use of both pre-trained or custom machine learning models. Though this step is more advanced it might be necessary depending on the nature of the images to be analyzed. The user can attempt it after they have acquired some experience and practice through conventional image analyses and might need to consult with a more experienced colleague, depending on the complexity of the task.

12. Install PyTorch, a machine learning framework, from its website.

△ **CRITICAL:** Installation differs depending on operating system (e.g., Windows, Mac, Linux), installation package (e.g., pip, Libtorch, Source), programming language (e.g., Python), and whether processing will be performed with the CPU or GPU (using CUDA). To properly install PyTorch see <https://pytorch.org/> and select the proper parameters for your device. A command will then be given, that when run from the terminal will install PyTorch.

13. Test several pre-trained machine learning based segmentation models.

△ **CRITICAL:** Due to the complexity and time requirement for training a custom machine learning segmentation model, several pre-trained models should be tested prior to using a custom model.

- a. Install and test Segment Anything for Microscopy (μSAM), a pre-trained model designed specifically for microscopy images.²⁵
 - i. Create and activate a virtual environment with Conda for μSAM (see preparation two).
 - ii. In the terminal with the Conda environment active, install 'micro_sam'.

```
>conda install -c conda-forge micro_sam
```

- iii. For automatic segmentation with μSAM, download the Jupyter notebook here: https://github.com/computational-cell-analytics/micro-sam/blob/master/notebooks/automatic_segmentation.ipynb and select the μSAM Conda environment as the Python kernel in the Jupyter notebook.

Note: The Jupyter notebook above will download and segment an example dataset μSAM.

- iv. To segment specific images, add the following code to the above Jupyter notebook.

```
>model_choice = "vit_b_lm"
>img = "path/to/image"
# generate mask
```

```
>prediction = run_automatic_instance_segmentation(img, ndim=2, model_type=model_choice)

# visualize masks

>plot_samples(image=raw, gt=labels, segmentation=prediction)
```

Note: If an error stating that the unexpected keyword argument 'amg' was received, resolve it by commenting out the lines mentioning 'amg' within the function 'run_automatic_instance_segmentation'.

- b. Install and test Medical Segment Anything Model (MedSAM), designed to distinguish foreground from background within a user-supplied bounding box for a wide variety of medical images.²⁴

△ **CRITICAL:** PyTorch must be configured for processing with GPUs via CUDA to use MedSAM. This means that a version of PyTorch with CUDA support is required. If GPUs are not available for processing via CUDA, then another model must be used.

- i. Install Git for the current operating system using instructions and links for download at <https://git-scm.com/install/>.

△ **CRITICAL:** Git installation varies between operating systems. On Windows, Git can be downloaded directly. On Mac, a package manager (e.g., homebrew) must be installed prior. Homebrew can be installed by running the prompt found at <https://brew.sh/> in the terminal.

- ii. Create a virtual environment with Conda for MedSAM (see preparation two).

Note: For proper organization and to avoid conflicting dependencies, it is recommended that each machine learning model and its dependencies are installed in separate Conda environments.

- iii. Open the Git bash terminal.

Note: The Git bash terminal can be opened in a similar manner to other terminals by entering "Git bash" in the system's search feature (see preparation one).

- iv. If running Conda in the Git Bash terminal for the first time, source Conda using the following commands.

```
# if using Windows

>export PATH="path/to/conda/dir/miniconda3/Scripts:$PATH"

# if using MacOS or Linux

>export PATH="path/to/conda/dir/miniconda3/bin:$PATH"

# on any operating system run following

>conda init bash
```

Note: On a Windows system, 'Scripts' should be included in the path. On MacOS or Linux systems, 'bin' should be included instead.

△ **CRITICAL:** The terminal should be closed and reopened after sourcing Conda in order for updates to occur.

- v. In the Git bash terminal, activate the newly created virtual environment and install PyTorch 2.0 within the environment (see step 12).
- vi. Navigate the directory where the model should be downloaded locally by running the command 'cd' followed by the path to the folder.

```
>cd path/to/folder/
```

Note: The directory should not be synced via cloud file systems (e.g., OneDrive, Dropbox) as this can interfere with GitHub version management functionality.

- vii. Using the git bash terminal, clone the MedSAM repository.

```
>git clone https://github.com/bowang-lab/MedSAM.
```

- viii. Run the following commands in the same Git bash terminal to install required packages.

```
>cd MedSAM
>pip install -e .
```

- ix. Download the current MedSAM model checkpoint (e.g., "medsam_vit_b.pth") at https://drive.google.com/drive/folders/1ETWmi4AiniJeWOt6HAsYgTjYv_fkgoN.

Note: Users should refer to MedSAM documentation for the most up-to-date model.

- x. Move the installation to the directory where it will be referenced.

```
>mv /path/to/model/file.pth /path/to/medsam /directory/work_dir/MedSAM/
```

- xi. Test if the installation is successful by running the following command from the MedSAM directory.

```
>python MedSAM_Inference.py
```

Note: If Error #15 is thrown, navigate to the directory miniconda3/envs/medsam/Library/bin and delete the file 'libiomp5md.dll'

- xii. To start segmenting, open the file 'tutorial_quickstart.ipynb' with the MedSAM Conda environment as a kernel.

Note: The file 'tutorial_quickstart.ipynb' is located in the base MedSAM directory that was cloned from GitHub.

- xiii. Within the Jupyter Notebook, the following chunk will open a prompt with the loaded image where objects will be segmented within manually drawn bounding boxes.

```
>img = "assets/img_demo.png"
>bbox_prompt_demo = BboxPromptDemo(medsam_model)
```

```
>bbox_prompt_demo.show(img)
>print(bbox_prompt_demo)
```

⚠ **CRITICAL:** To segment images other than the demo, replace the image path.

- xiv. After adding segmentations, click the 'save' button to save the segmentation as a png file and read the segmentation.png file into Python as a numpy array.

```
>from PIL import Image
>png_img = Image.open('path/to/png/file')
>png_img_greyscale = png_img.convert("L")
>png_array = np.array(png_img_greyscale)
```

Note: When saving, both the annotated image and the segmentations alone will be saved as two separate .png files in the current directory. The segmentation will be saved as 'seg.png'

⚠ **CRITICAL:** If segmenting multiple images simultaneously, the file name of the saved segmentations should be manually changed before starting the next segmentation to avoid overwriting files.

14. Install and import necessary modules from PyTorch, a machine learning framework, and torchvision, a computer vision extension for PyTorch. ^{12,13}

```
# install required libraries
>!pip install -q torch torchvision

# import modules for UNET model
>import torch
>import torch.nn as nn
>import torch.nn.functional as F
>from torch.utils.data import Dataset, DataLoader
>from torchvision import transforms
>from PIL import Image
>import os
>from tqdm import tqdm
>import numpy as np.
>import matplotlib.pyplot as plt # for plotting segmentations
```

Note: This example shows a custom UNet model optimized to identify proteinaceous inclusions. Compared to pre-trained models, such as Cellpose, a custom trained UNet model allows for the most control.

Note: There are several options for convolutional neural networks (CNNs) used for image segmentation, including UNet, UNet++, and μ SAM. UNet is a commonly used model for image segmentation and will be used as the base architecture for an example.¹⁹

15. Generate a set of manually annotated images for model training and validation. Example datasets are listed in [Table 1](#).

△ CRITICAL: Annotations should be made by trained annotators using a consistent definition of the objects of interest. Annotations can be made on raw images using platforms that create mask label maps, such as Supervisely.

Note: A UNet model requires a large number of images for training. A number in the low thousands is recommended for effective results, though this varies greatly between models and the complexity of segmentation tasks. Smaller datasets (e.g., 200-400) can be used to test the performance of models, or on occasion can fully train models for simple tasks. A general rule of thumb is using 70% of an available dataset for training and the other 30% for testing the model. A Segment Anything Model UNet (SAM-UNet) model is recommended for image segmentation in the absence of a large training dataset as it has already been pre-trained on a very large dataset (see step 13).

- a. Store images and associated masks with the same name in sibling directories within a directory for the dataset.

△ CRITICAL: This dataset directory should be stored within the project folder (i.e., the working directory of the python script). In this example, images and their masks are stored as .png image files.

- b. Define methods for loading images and formatting for UNET interpretability.

```
>def load_16bit_image(image_path):
    """
    Load an image as a tensor
    Parameters:
    image_path (str): file path to the image
    Returns:
    tensor (Tensor): a tensor with the image data and dimensions
    """
    image = Image.open(image_path) # mode is 'I;16'
    np_image = np.array(image).astype(np.float32)
    # Normalize to [0, 1] using max possible 16-bit value
    np_image /= 65535.0
    # Add channel dimension and convert to tensor
    # shape: (1, H, W)
    tensor = torch.from_numpy(np_image).unsqueeze(0)
```

Note: The UNET model requires pixel values range from 0 to 1, so the original range of 1 to 16 from .png files requires normalization. Images are also converted to grayscale.

- c. Define a class structure for the annotated image dataset to ease and organize the process of reading and reformatting images.

```
>class GrayscaleSegmentationDataset(Dataset):
    """
    Describes the dataset of images to read
    Attributes:
    image_dir (str): file path to the image directory
    mask_dir (str): file path to the mask directory
    image_files (list): list of all image file names
    mask_files (list): list of all mask file names
    patch_shape (int, int): dimensions for rectangular patch
    """
    def __init__(self, image_dir, mask_dir, patch_shape=(662, 662)):
        """
        Initialize a new Dataset object
        Parameters are each attribute of the class
        """
        self.image_dir = image_dir
        self.mask_dir = mask_dir
        self.image_files = sorted(os.listdir(image_dir))
        self.mask_files = sorted(os.listdir(mask_dir))
        self.patch_shape = patch_shape
    def __len__(self):
        """ retrieve the number of image files """
        return len(self.image_files)
    def __getitem__(self, idx):
        """ Read a single image from the dataset as a tensor """
        image_path = os.path.join(self.image_dir, self.image_files[idx])
        mask_path = os.path.join(self.mask_dir, self.mask_files[idx])
        # Load 16-bit image without conversion
        image_tensor = load_16bit_image(image_path)
        # Load and resize mask
        mask = Image.open(mask_path).convert("L")
        mask_tensor = transforms.PILToTensor()(mask).squeeze(0).long()
```

```
# Convert all non-zero values to 1
mask_tensor = (mask_tensor > 0).long()
return image_tensor, mask_tensor
```

Note: The patch shape should be fit to the patch shape of the images. A patch is a small section (often a square) of the larger image that is processed independently of other patches. In machine learning, the algorithm for cleaning and extracting information from a total image is performed on each individual patch, rather than applying the algorithm to the entire image at once. This helps the algorithm better identify features of interest.

16. Define the class structures for the double convolution module for each encoder and decoder block.

```
>class DoubleConv(nn.Module):
    """
    Module that performs convolution on itself twice
    Attributes:
    in_c (int): number of channels the current module has
    out_c (int): number of output channels to produce
    """
    # initialization
    def __init__(self, in_c, out_c):
        """ Initialize a module with a method for double convolution with given number of input and
        output channels """
        super().__init__()
        self.double_conv = nn.Sequential(
            nn.Conv2d(in_c, out_c, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_c),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_c, out_c, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_c),
            nn.ReLU(inplace=True),
        )
    def forward(self, x):
        """ Run the double convolution on itself """
        return self.double_conv(x)
```

Note: Each double convolution module consists of two 3x3 convolutional layers, batch normalization, and rectified linear unit (ReLU) activation.

17. Define the class structure for the custom UNet model architecture.


```
>class UNet(nn.Module):

    """
    UNet image segmentation model that includes four sequential downsampling blocks with double
    convolution (down 1 through 4) followed by a double convolution to 512 channels (mid). Bilinear
    interpolation is used for upsampling through four blocks (up 1 through 4), each followed by a
    skip connection and double convolution that halves the number of channels (conv 1 through 4).
    The final convolution applies the extracted features to the number of channels provided.

    Attributes:

    down(1-4): downsample the given module and double the number of channels using double
    convolution

    mid: double the number of channels up to 512

    up(1-4): upsample a module to twice its previous size using bilinear interpolation

    conv(1-4): Use double convolution to merge the previous block and a skip connection, halving
    the number of feature channels from the previous block.

    """

    def __init__(self, n_classes):
        """
        Initialize a UNet model object

        Parameters:
        n_classes (int): the number of classes for segmentation

        """
        super().__init__()

        self.down1 = DoubleConv(1, 32)
        self.down2 = DoubleConv(32, 64)
        self.down3 = DoubleConv(64, 128)
        self.down4 = DoubleConv(128, 256)
        self.mid = DoubleConv(256, 512)

        # Use Upsample instead of ConvTranspose2d
        self.up4 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        self.conv4 = DoubleConv(512 + 256, 256)

        self.up3 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        self.conv3 = DoubleConv(256 + 128, 128)

        self.up2 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        self.conv2 = DoubleConv(128 + 64, 64)

        self.up1 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        self.conv1 = DoubleConv(64 + 32, 32)

        self.final = nn.Conv2d(32, n_classes, kernel_size=1)

    def forward(self, x):
        """
        Send image through each step of the UNet model

```

Parameters:

`x (Tensor)`: image tensor to send through the model

Outputs:

`self.final (Tensor)`: a segmentation map of the given image

"""

```

    d1 = self.down1(x)
    d2 = self.down2(F.max_pool2d(d1, 2))
    d3 = self.down3(F.max_pool2d(d2, 2))
    d4 = self.down4(F.max_pool2d(d3, 2))
    mid = self.mid(F.max_pool2d(d4, 2))
    u4 = self.up4(mid)
    if u4.size() != d4.size():
        u4 = F.interpolate(u4, size=d4.shape[2:], mode='bilinear', align_corners=True)
    u4 = self.conv4(torch.cat([u4, d4], dim=1))
    u3 = self.up3(u4)
    if u3.size() != d3.size():
        u3 = F.interpolate(u3, size=d3.shape[2:], mode='bilinear', align_corners=True)
    u3 = self.conv3(torch.cat([u3, d3], dim=1))
    u2 = self.up2(u3)
    if u2.size() != d2.size():
        u2 = F.interpolate(u2, size=d2.shape[2:], mode='bilinear', align_corners=True)
    u2 = self.conv2(torch.cat([u2, d2], dim=1))
    u1 = self.up1(u2)
    if u1.size() != d1.size():
        u1 = F.interpolate(u1, size=d1.shape[2:], mode='bilinear', align_corners=True)
    u1 = self.conv1(torch.cat([u1, d1], dim=1))
    return self.final(u1)

```

Note: This UNet model uses bilinear interpolation instead of the default two-dimension convolution for map upsampling. Bilinear interpolation is more memory efficient and prevents checkerboard artifacts. After upsampling, a skip connection is used to integrate information from each layer of the encoder and decoder. This is then passed to a double convolution block for feature refinement. Decoder feature maps are interpolated to match the size of their paired encoder block prior to concatenation. Finally, the output segmentation label is generated from a 1X1 convolution of the decoder block's output onto a binary classification.

18. Train the UNet model using manually annotated training images.

```

# define function to train the model
>def train_model(image_dir, mask_dir, num_classes=2, num_epochs=25):

```

```
"""
Trains a custom UNet segmentation model given a set of raw images and their manually segmented
counterparts

Parameters:

image_dir (str): file path to the directory of raw images

mask_dir (str): file path to the directory of manually segmented masks

num_classes (int): number of discrete object groups to classify segmented objects as

num_epochs (int): number of times to pass the training data set through the training algorithm,
allowing the model to update

No output in Python environment but saves model as a .pth file
"""

# load the dataset
dataset = GrayscaleSegmentationDataset(image_dir, mask_dir)
dataloader = DataLoader(dataset, batch_size=1, shuffle=True)

# attempt to run model on GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = UNet(n_classes=num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for images, masks in tqdm(dataloader):
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        loss = criterion(outputs, masks)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    total_loss += loss.item()
    print(f"Epoch{epoch+1}/{num_epochs}, Loss: {total_loss/len(dataloader):.4f}")

# save the model
torch.save(model.state_dict(), "unet_light_colab.pth")
print("Training complete. Model saved.")

# run the function to train the model
>train_model(path_to_images, path_to_masks, num_classes=2, num_epochs=25)
```

△ **CRITICAL:** Supply the function with the path to the image files, the path to the mask files.
The number of classes should be kept at 2 for binary segmentations. The number of epochs

may be altered when optimizing the model. After training the model is saved as a .pth file that will allow for future loading and image segmentation.

Note: This process is often highly computationally intensive and can take an extended amount of time to compute. Using a GPU for training can significantly shorten the time required to train and use a machine learning model, particularly for large datasets. (For information on using PyTorch with GPUs, see step 12).

19. Use the trained model to segment images.
 - a. Load the model from the previously saved training.

```
>model = UNet(n_classes=2)

>model.load_state_dict(torch.load("UNET_light_colab.pth", map_location=torch.device('cuda'
if torch.cuda.is_available() else 'cpu'))))

>model.eval()
```

Note: If GPUs are configured with CUDA this will set the model to run using the GPUs. Otherwise, the model will run using the host computer's CPU.

- b. Run the model on a select collection of images, plotting the segmented image along with the original.

```
# define function to load and segment each image

>def predict_and_show_folder(image_folder, model, patch_shape=(656, 656), max_images=100):
    """
    Run a trained Unet model to segment a folder of images

    Parameters:
    image_folder (str): file path to a folder with raw images
    model (module): previously trained and loaded model
    patch_shape (int, int): dimensions for rectangular patch
    max_images (int): maximum number of images to run at a time, influenced by computational re-
    straints from hardware.
    """

    # load the model
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = model.to(device)
    model.eval()

    # sort the image files to be loaded
    image_files = sorted([
        >f for f in os.listdir(image_folder)
        >if f.lower().endswith(('.png', '.jpg', '.jpeg', '.tif', '.tiff'))
    ])

    # load and generate segmentations with the trained model
```

```
for i, filename in enumerate(image_files[:max_images]):
    image_path = os.path.join(image_folder, filename)

    # Load and preprocess 16-bit image
    image_tensor = load_16bit_image(image_path)
    input_tensor = image_tensor.unsqueeze(0).to(device)

    # Predict
    with torch.no_grad():
        output = model(input_tensor)
        pred = torch.argmax(output, dim=1).squeeze().cpu().numpy()

    # Convert normalized image to 2D for plotting
    image_np = image_tensor.squeeze(0).numpy()

    # plot the original image and segmentation
    fig, ax = plt.subplots(1, 2, figsize=(10, 5))
    ax[0].imshow(image_np, cmap="gray", vmin=0, vmax=1)
    ax[0].set_title(f"Normalized Original: {filename}")
    ax[0].axis("off")
    ax[1].imshow(pred, cmap="jet")
    ax[1].set_title("Predicted Segmentation")
    ax[1].axis("off")

    plt.tight_layout()
    plt.show()

# run the function
>predict_and_show_folder(path_to_image_folder, model)
```

△ **CRITICAL:** The following should be supplied to the defined function: a path to the image folder, the trained and loaded UNet model, the patch shape of the images, and the maximum number of images to process. The patch size may be adjusted to improve efficiency.

Note: The parameter 'max_images' can be adjusted depending on computational restrictions imposed by hardware, particularly space in RAM. Greater RAM capacity will allow for more images to be segmented with each execution.

Quantification of biological parameters

⌚ **Timing:** ~1 h

This step outlines individual measurements and features that can be quantified from segmented images. Examples of measurements used in the accompanying manuscript are also provided.

20. Select what feature or features of the components to measure. Described below are common measurements.

- a. Counting the number of nuclei in the image.

```
>nuclei_labeled = skimage.measure.label(nuclei_mask[nuclei_mask>0])
>nuclei_count = len(np.unique(nuclei_mask))
```

Note: Nuclei are used as an example, however this also applies to counting other objects as well. This is accomplished by counting the number of non-overlapping segmented objects in the mask and retrieving the number of nuclei.

- b. Cell counting with Cellpose segmentation.
 - i. Import or install Cellpose and segment the image using a Cellpose model (see step 11).
 - ii. Retrieve the number of cells after Cellpose segmentation.

```
>cell_count = cellpose_masks.max()
```

- c. Measuring surface area by counting the number of pixels segmented. This is accomplished by summing the mask of the component to retrieve the size of the component in pixels.

```
>surface_area = np.sum(mask)
```

- d. Measuring the intersection between two components in different channels to examine how they colocalize.

```
>intersection = red_channel_mask & green_channel_mask
```

Note: This is accomplished by applying a bitwise ‘logical and’ operation to produce a mask with pixels segmented in both components.

Note: Each component should be segmented individually by applying preprocessing algorithms to each channel.

- e. Mean fluorescence intensity (MFI), the average fluorescence of each segmented pixel. This is accomplished by computing the mean fluorescence of all pixels in the original image on which a binary mask generated after segmentation is overlaid.

```
>mfi = np.mean(original_image[binary_mask > 0])
```

- f. Shape analysis, including measures of circularity and elongation.
 - i. Segment the component of interest.
 - ii. Label each separate object in the mask.

```
>image_labeled = skimage.measure.label(binary_mask)
```

- iii. Retrieve each object’s properties as a list of attributes using scikit-image.

```
>props = skimage.measure.regionprops(image_labeled)
# assign the area of the first object to a variable
>object_area = props[0].area
```

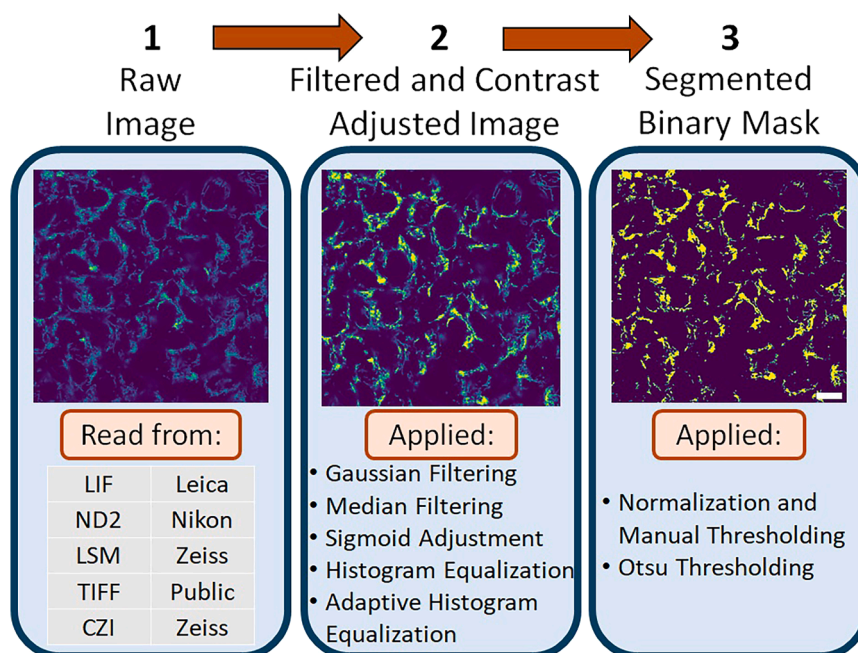


Figure 1. Pipeline for Image Preprocessing

Steps include denoising with a filter, contrast adjustment, generating a threshold for segmentation, and segmenting the image by generating a binary mask. In some scenarios, denoising or contrast adjustment may not be required. Scale bar = 10µm.

Note: Properties include the number of pixels (i.e., area), the length of major and minor axes, the coordinates of the bounding box (i.e., the smallest possible rectangle that contains the whole object). Properties are returned as a list where each element corresponds to a labelled object. Individual object properties can be called using the associated key.

Table 1. Publicly available image and segmentation datasets

Name	Identifier	Description	Data scale	Covered task types	Access channels
Neurips Segmentation Challenge 2022 ²⁰	Ma et al., 2024	102 labeled images with a variety of cell morphology, stains, and microscopy techniques	Unscaled	Instance Segmentation, Weakly supervised segmentation	Single access channel for each image differing based on stain or microscope technique used.
Kaggle 2018 Science Bowl ²¹	Caicedo et al., 2019	670 images of nuclei labeled with a variety of stains	Images of various sizes and scales	Instance Segmentation	Single access channel for each image differing based on stain or microscope technique used.
Broad Bioimage Benchmark Collection ²²	Ljosa et al., 2012	54 distinct datasets, spanning various organisms, microscopy techniques, and segmentation tasks.	Intensity is unscaled, image size varies between individual datasets.	Binary segmentation, instance segmentation,	Number of channels differs between datasets. Many contain a single channel while some fluorescent images have multiple.
Cell Tracking Challenge ²³	Maška et al., 2023	Many separate datasets are available, showing cells moving using fluorescent or brightfield microscopy	Intensity is unscaled, image size varies between individual datasets.	Instance segmentation, temporal segmentation	Single access channel for each image differing based on stain or microscope technique used.
Cellpose Training Data ¹⁴	Stringer et al., 2021	549 images of cells or similarly shaped objects spanning a variety of microscopy techniques	Intensity is unscaled	Instance segmentation	Mix of single access channel images showing only cytoplasm or two access channels (cytoplasm and nuclei) for some fluorescent images.

- iv. Compute desired measures of shape using the retrieved properties retrieved. Some examples are provided.

1. Compute elongation, the length of the minor axis compared to the major axis of the object.

```
>elongation = props[0].axis_minor_length / props[0].axis_major_length
```

2. Compute circularity, how well the object matches a perfect circle of the same area.

```
>circularity = 4 * np.pi * (props[0].area / props[0].perimeter**2)
```

Note: Shape analysis is performed on individual (non-continuous) objects within the binary mask. Properties of each non-continuous object will each be stored as an object in a list, demonstrated in the code examples below.

Optional: Apply measurements to time-course or z-stack image types by iterating over an additional axis of the image array (e.g., measuring each image while iterating over the Z axis).

21. Combine various measurements based on image types and research questions. This may not be required, however specific image analysis examples (including specific treatments) are provided in [Table 2](#) and [Figure 2](#) for reference. For more detail on the execution of these experiments, refer to the accompanying manuscript.¹
22. Store results from each image in a pandas dataframe.
 - a. Prior to iterating over each image file, initialize a list to store individual file dataframes.

```
>df_list = [] # list to store each file's dataframe
```

- b. Create and store recorded measurements in a pandas dataframe organized by each measured object and image, including cells, time periods, or slices. Append the dataframe to the list of dataframes.

```
# write data from each file to separate dataframe
>example_df = pd.DataFrame({
    'File': file_path,
    'Cell': cell_number, # if segmenting individual cells
    'Time': time_point, # if timecourse experiment
    'Slice': slice_number, # if measuring z-stack
    'Measure_1': measure_1_data,
    'Measure_2': measure_2_data
})
# append the dataframe to the list of dataframes
>df_list.append(example_df)
```

- c. After every image file has been processed, combine each dataframe in the list into a single dataframe


```
>all_data = pd.concat(df_list, ignore_index=True)
```

Visualizing images and exporting data

⌚ Timing: 15 min

During and after each image is processed, it is important to print the images to ensure that each step, particularly segmentation, has occurred successfully. This details how single images or multiple images can be printed as multiplots using matplotlib to facilitate visual inspection. If processing and measurement is satisfactory, the results can be saved as a .xlsx file type for future analysis. The final step describes how to write the measurements, organized into a pandas dataframe, as a .xlsx file type.

23. Print images (stored as numpy arrays) using matplotlib during the processing pipeline for visual inspection.
 - a. Print single images by supplying a two-dimensional array (i.e., x and y axes) to the function 'plt.imshow()'.¹¹

```
>plt.imshow(image_2d)
>plt.show()
```

Note: The function 'show()' from pyplot in matplotlib should be called to display any previously generated images.

⚠ CRITICAL: Only two-dimensional arrays can be printed in a single plot. To print multiple channels or slices of a Z-stack, multiple plots must be created.

- b. Print multiple images simultaneously as sections of a subplot.

```
>fig, ax = plt.subplots(1, 3, figsize=(15, 9))
>ax[0].imshow(image_1)
>ax[1].imshow(image_2)
>ax[2].imshow(image_3)
>ax[2].set_title('Blue - Hoechst') # give plot a title
>ax[2].axis('off') # optional, turn off plot axes
>plt.show()
```

Note: Adjusting the values of the first and second parameters controls the number of plots vertically and horizontally respectively. Adjusting the first and second values supplied to 'fig-size' will alter the width and height respectively of the subplot as a whole.

24. Save the collected measurements, organized in a pandas dataframe, as an .xlsx file.¹⁰ Troubleshooting 5.

```
>dataframe.to_excel(file_path.xlsx)
```

Table 2. Examples of base measurement applications from Eubanks et al., 2025

Measurement	Description	Measurement and metadata used	Key libraries and functions
Inclusion Counting	Inclusions are segmented and labeled, then counted. The number of cells is counted from Cellpose segmentation. The ratio of the number of inclusions to number of cells is computed.	Object counting, Cellpose segmentation	Cellpose, Skimage (label)
Lipid droplet-Inclusion Intersection	Lipid droplets and inclusions are both segmented in separate channels. An intersection is then taken between both masks. The surface area of the intersection is standardized to both a percent of the lipid droplet surface area and the percent of the inclusion surface area.	Surface area, Intersection	NumPy (logical_and, sum)
Mitochondrial Membrane Potential Z-Stacks	Mitochondria, imaged by staining cells with tetramethylrhodamine methyl ester (TMRM), are segmented and the mean fluorescence intensity is collected from each slice of a z-stack. The mean fluorescence intensity is then averaged across all slices.	Mean Fluorescence Intensity	NumPy (mean)
Propidium Iodide Cellular Viability	Cells, stained with Hoechst and propidium iodide (PI), are segmented and labeled using Cellpose. PI staining is segmented in a separate channel. The intersection between each cell and PI is computed, and the ratio of cells positive for PI intersection is compared after normalization to the total count of cells.	Cellpose segmentation, Intersection	Cellpose, NumPy (logical_and, sum)
Inclusion Shape Analysis	Inclusions are segmented, labeled, and shape features are derived from each individual object. Various measurements of shape (e.g., circularity, elongation) are used to categorize the inclusion into one of several groups by shape.	Shape analysis	Skimage (label, regionprops)

Note: The dataframe may be written to other file types by calling a similar function, such as 'to_csv()' in the pandas library.¹⁰

EXPECTED OUTCOMES

This protocol describes the process of reading, preparing, segmenting, and measuring images of biological specimens with Python and various libraries. While the specific steps taken for each stage may vary based on the images and experiment, the protocol will produce a pandas dataframe with the desired measurements organized by image, slice, time point, or other variables. We detail how this dataframe may be written to various file types for future analysis in Python or other software. An example dataframe can be seen in [Table 3](#).

LIMITATIONS

Depending on the quality of each image in a set, some images will not segment properly. In our experience, roughly eighty percent of a set will segment well and provide useful data. Additionally, Python does not include a graphical user interface by default unlike other software (e.g., ImageJ, ZEN). Subsequently, manual annotation or segmentation of images requires additional programming in Python. Manual annotations can be added to images in Python through libraries such as Napari, though this requires the installation of additional modules and further programming.²⁶ In some cases, manual annotations can be read along with the image data, though this requires additional processing (see step 5).

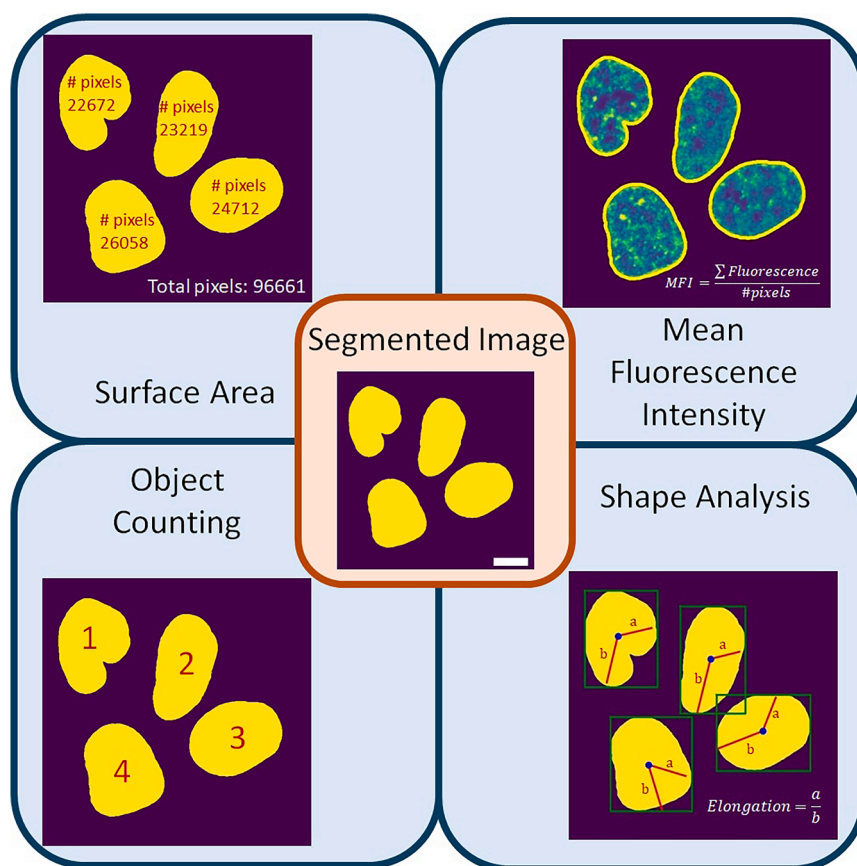


Figure 2. Outline of Basic Measurements

Several measurements can be derived from the segmented image. Surface area (top left) is the sum of segmented pixels. Mean fluorescence intensity (top right) is the average value for each segmented pixel. Counting (bottom left) can be accomplished through object labeling in scikit-image or through cell segmentation with Cellpose. Shape analysis (bottom right) includes measures of the elongation and circularity of each object. Scale bar = 5μm.

This procedure can also be very computationally intensive, particularly for large datasets or images containing many slices or time points. Certain filtering algorithms, including Gaussian filtering and local thresholding algorithms increase the time required to process images. Machine learning approaches are also computationally intensive. This is especially true if GPUs are not available for training and using machine learning models, as GPUs can process the many small commands required for machine learning at a faster rate than a CPU. This difference in using a GPU can reduce the time required for training and use of a machine learning model (possibly by hours or days) depending on the size of the dataset. Additionally, image file types are often large and storing many images may require a substantial amount of memory on both the hard drive and RAM.

Table 3. Example dataframe output from image processing and object measurement

image	SA_green	SA_subtraction	subtraction_green_ratio
Image_1.czi	5217273	10055	0.001927252
Image_2.czi	4931872	8374	0.001697935
Image_3.czi	3744610	32023	0.008551758
Image_4.czi	4240818	9848	0.002322194
Image_5.czi	3600217	9495	0.002637341

SA, surface area.

TROUBLESHOOTING

Problem 1

Python is not recognized when calling from the terminal ([before you begin](#) step 1).

Potential solution

- Call Python with 'py' or 'python3' instead of 'python'. Different operating systems will respond differently to each due to the definition of computer path variables upon Python's installation. Mac and Linux come with Python version 2 preinstalled, and will only recognize the command 'python3'.

Problem 2

When reading image files, .czi files cannot be read properly (Step 4).

Potential solution

- Apply a different approach for reading .czi files using a different package. If not already installed, install and load the library `czifile`.¹⁶

```
>!pip install czifile
>import czifile
```

- Read the .czi file into Python as a czi image object.

```
>czi_image = czifile.imread(czi_file_path)
```

- Print the abbreviation and size of each dimension of the image object. A list of dimensions and their abbreviations are listed in [Table 4](#).

```
>print(czi_image.axes)
>print(czi_image.shape)
```

- Convert the czi image object to a numpy array and squeeze (remove any dimension with size one). This will leave only pixel values and additional image information (e.g., slice, time).

```
>czi_array = czi_image.asarray()
>czi_array_squeezed = np.squeeze(czi_array)
```

Problem 3

Poor segmentation of images when using traditional methods (Steps 6 through 10). Different contrast adjustment and thresholding methods allow for better segmentation of images. Poor segmentation may identify too much or too little of the image as the foreground, with examples shown in [Figure 3](#).

Table 4. Abbreviations for data stored in a.czi object type when read with the library czifile¹⁶

Abbreviation	Data
X	X-axis
Y	Y-axis
C	Channel
Z	Slice, or Z-axis
T	Time Point
R	Rotation
S	Scene
I	Illumination
B	Block
M	Mosaic Tile Index
H	Phase

Potential solution

- If applicable, change the method of filtering and contrast adjustment to better fit the image. For example, histogram equalization will often expand the region which is segmented, while local methods of contrast adjustment will make the same thresholding algorithm (e.g., Otsu) more selective.
- Alter the method of thresholding. For sets of images with less variability, normalizing images and segmenting with a manually selected threshold often works, however with images that vary in contrast (e.g., slices of a z-stack) more dynamic algorithms such as Otsu are often preferred.

Problem 4

Images are poorly segmented using Cellpose (Step 11). This includes Cellpose disregarding parts of the cell, cells being merged together, or empty spaces being identified as cells. This can occur when images include bright round objects, such as a proteinaceous inclusion, which may be disregarded from the cell or counted as separate cells. Examples of segmentation are shown in [Figure 4](#).

Potential solution

- Adjust the diameter parameter when evaluating the image. Depending on the level of magnification, diameter (measured in pixels) should be adjusted to fit the size of individual cells. To account for variation in cell size between images, it is often advantageous to sweep across a range of diameters and apply the smallest value that segments cells. This involves using the model to segment at a variety of diameters. Code for this is in a two dimensional example shown below.

```
# iterate over a range of diameters
>for diameter in range(200, 700, 25):
    masks, _, _ = model.eval(
        channel_data,
        diameter=diameter,
        channels=channels,
    )
    labeled = label(masks)
    if np.max(labeled) > 0:
        return labeled
```

```
# return array of zeros if nothing segments
print("No cells found.")
return np.zeros_like(green_channel)
```

- Use different filtering and contrast adjustment techniques specifically to enhance the outline of cells. Filters that emphasize large differences in fluorescence and minimize local details, such as histogram equalization, may be the most successful.
- Include a nuclear channel in addition to a cytoplasmic one, which improves the performance of Cellpose.

Problem 5

Cannot write a dataframe as a .xlsx file (Step 22). Running the line below in a Jupyter Notebook should return openpyxl and the module version. If not, openpyxl should be installed.

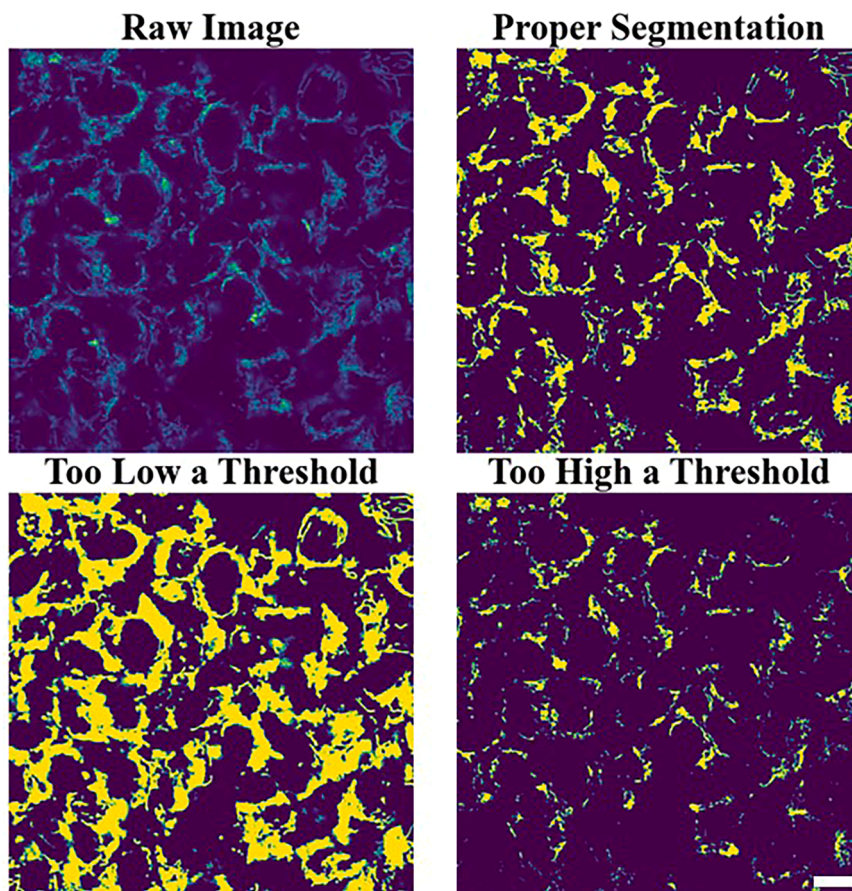


Figure 3. Examples of Proper and Improper Image Segmentation

Segmentation can include too much or too little of the image depending on the threshold. Contrast adjustment methods or changing the thresholding method may improve segmentation. Binary masks should be inspected for each segmented image to ensure that segmentation occurred properly. Scale bar = 5 μ m.

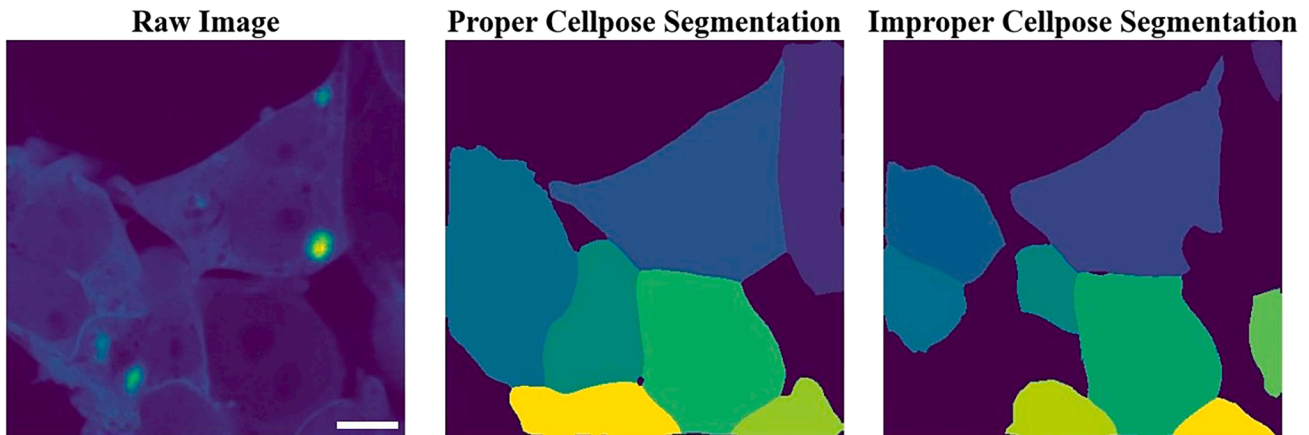


Figure 4. Examples of Proper and Improper Cell Segmentation and Labeling using Cellpose

When labeling cells with Cellpose, individual cell labels may divide single cells, identify background spaces as cells, or miss existing cells. Cellpose model parameters should be adjusted to be fit each given experiment, as cell sizes and shapes may change. Scale bar = 5 μ m.

Potential solution

- Verify that the library openpyxl is installed. Running the line below in a Jupyter Notebook should return details on openpyxl. If not, openpyxl should be installed. If running from the terminal the exclamation point should be removed.

```
>!pip show openpyxl
```

- If openpyxl is not installed, install by entering the following command in your terminal or Jupyter Notebook.

```
>!pip install openpyxl
```

Problem 6

Complementary images (i.e., separate images used to generate a single measure, applicable for example for colocalization analysis) have different lengths of the x or y axes. This will create a problem when masking one image with another, used in colocalization analyses (Step 9).

Potential solution

- Resize the mask generated from another image to match the shape of the measured image. This will alter the scale of the pixels and should only be used if the image is being adjusted to fit the scale of pixels in another image.

```
>mask_resize = skimage.transform.resize(mask, other_image.shape, False)
```

- If images are the same scale but the images are not the same size, add padding (i.e., solid colored borders) to the edges of the smaller image instead of rescaling.

```
>mask_resize = np.pad(masks, ((top_border, bottom_border), (left_border, right_border)))
```


RESOURCE AVAILABILITY

Lead contact

Requests for clarification or more information regarding the resources and data included in this protocol should be sent to the lead contact, Eleanna Kara (ek851@rwjms.rutgers.edu).

Technical contact

Questions regarding the execution of this protocol should be directed to the technical contacts, Benjamin Sacks (bsacks@mgm.harvard.edu) and Eleanna Kara (ek851@rwjms.rutgers.edu).

Materials availability

This study did not generate new unique materials.

Data and code availability

- All original code and data used to generate the figures have been deposited on Zenodo: <https://doi.org/10.5281/zenodo.17757130>.
- Original code and data used for figure generation are also available as a publicly repository at https://github.com/eleannakara/Kara-Lab/tree/main/image_analysis_methods_2025.

ACKNOWLEDGMENTS

We acknowledge the following funding for student research fellowships: NIH R25 grant (R25NS105143) to Rutgers NeuroSURP summer student program (B.S. and A.M.), AOA Carolyn L. Kuckein Student Research Fellowship (E.E.), Jon Kent Cooke foundation graduate scholarship (J.C.), Aresty summer science research fellowship (N.R.S.), and Rutgers Health Center for Biomedical Informatics & Health Artificial Intelligence (BMIHA) summer research internship (S.J.). We are grateful to Dr. Zainab Tanvir for providing us access to .lif images and to Katelyn VanderSleen, Neha Patel, Philip Socha, and Sanjana Archakam for technical assistance during early stages of this project.

AUTHOR CONTRIBUTIONS

B.S., J.M., Y.H., S.J., R.K., and A.G. wrote the code; E.E., E.G., J.C., B.S., N.R.S., A.D., N.R., A.M., J.M., and E.K. undertook the experiments; E.G., N.R.S., A.D., N.R., and A.M. annotated images; A.C. provided critical advice; B.S. wrote the manuscript; E.K., A.C., and B.S. edited the manuscript; E.K. acquired funding; E.K. developed the study concept and design; and E.K. supervised the study.

DECLARATION OF INTERESTS

E.K. is a member of the EMBO Scientific Exchange Grants Advisory Board.

REFERENCES

- Eubanks, E., VanderSleen, K., Mody, J., Patel, N., Sacks, B., Farahani, M.D., Wang, J., Elliott, J., Jaber, N., Akçimen, F., et al. (2025). Increased burden of rare risk variants across gene expression networks predisposes to sporadic Parkinson's disease. *Cell Rep.* 44, 115636. <https://doi.org/10.1016/j.celrep.2025.115636>.
- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., et al. (2016). In Jupyter Notebooks – a Publishing Format for Reproducible Computational Workflows. F. Loizides and B. Schmidt, eds. (IOS Press), pp. 87–90. <https://doi.org/10.3233/978-1-61499-649-1-87>.
- Gohlke, C. (2022). tiffio: Read and write TIFF files. Version 2025.1.10 (Zenodo). <https://doi.org/10.5281/zenodo.14630101>.
- Lambert, T. (2022). nd2 0.0.1rc5 : Yet another nd2 (Nikon NIS Elements) file reader. (Version 0.0.1rc5. <https://github.com/tlambert03/nd2>.
- Negretti, N. readlif: Fast Leica LIF file reader written in python. Version 0.6.5. <https://github.com/nimne/readlif>.
- Watkins, P. (2019). aicspylib: A python module and a python extension for Zeiss (CZI/ ZISRAW) microscopy files. Version 3.3.1. <https://github.com/AllenCellModeling/aicspylib>.
- Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., et al. (2020). Array programming with NumPy. *Nature* 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>.
- van der Walt, S., Schönberger, J.L., Nunez-Iglesias, J., Boulgogne, F., Warner, J.D., Yager, N., Gouillart, E., and Yu, T.; scikit-image contributors (2014). scikit-image: image processing in Python. *PeerJ* 2, e453. <https://doi.org/10.7717/peerj.453>.
- Haase, R., Lampert, M., Witz, G., Korten, T., Ankenbrand, M.J., Kugler, E., Zoccoler, M., amgfernandes, Taylor, S., and Ray, A. (2024). haesleinhuepf/BiolImageAnalysisNotebooks: gpt-2024.1.6 (Zenodo). <https://doi.org/10.5281/zenodo.10465773>.
- The pandas development team (2026). pandas-dev/pandas: Pandas (v3.0.0) (Zenodo). <https://doi.org/10.5281/zenodo.1832852>.
- Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment. *Comput. Sci. Eng.* 9, 90–95. <https://doi.org/10.1109/MCSE.2007.55>.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Preprint at arXiv. <https://doi.org/10.48550/arXiv.1912.01703>.
- maintainers, T., and contributors. (2016). TorchVision: PyTorch's Computer Vision library (GitHub). <https://github.com/pytorch/vision>.
- Stringer, C., Wang, T., Michaelos, M., and Pachitariu, M. (2021). Cellpose: a generalist algorithm for cellular segmentation. *Nat. Methods* 18, 100–106. <https://doi.org/10.1038/s41592-020-01018-x>.
- Murray, A., van Kemenade, H., wiredfool, Clark, J.A., Karpinsky, A., Baranović, O., Gohlke, C., Yay295, Dufresne, J., and Brett, M.. python-pillow/Pillow: 11.1.0 (11.1.0). Zenodo. <https://doi.org/10.5281/zenodo.14586647>.

16. Gohlke, C. (2025). cgohlke/czifile: v2019.7.2.1 (v2019.7.2.1) (Zenodo). <https://doi.org/10.5281/zenodo.14948582>.
17. Braun, G.J., and Fairchild, M.D. (1998). Image lightness rescaling using sigmoidal contrast enhancement functions. In *Color Imaging: Device-independent Color, Color Hardcopy, and Graphic Arts IV (SPIE)*, pp. 96–107. <https://doi.org/10.1117/12.334548>.
18. Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. *IEEE Trans. Syst. Man Cybern.* 9, 62–66. <https://doi.org/10.1109/TSMC.1979.4310076>.
19. Ronneberger, O., Fischer, P., and Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. Preprint at arXiv. <https://doi.org/10.48550/arXiv.1505.04597>.
20. Ma, J., Xie, R., Ayyadury, S., Ge, C., Gupta, A., Gupta, R., Gu, S., Zhang, Y., Lee, G., Kim, J., et al. (2024). The multimodality cell segmentation challenge: toward universal solutions. *Nat. Methods* 21, 1103–1113. <https://doi.org/10.1038/s41592-024-02233-6>.
21. Caicedo, J.C., Goodman, A., Karhohs, K.W., Cimini, B.A., Ackerman, J., Haghighi, M., Heng, C., Becker, T., Doan, M., McQuin, C., et al. (2019). Nucleus segmentation across imaging experiments: the 2018 Data Science Bowl. *Nat. Methods* 16, 1247–1253. <https://doi.org/10.1038/s41592-019-0612-7>.
22. Ljosa, V., Sokolnicki, K.L., and Carpenter, A.E. (2012). Annotated high-throughput microscopy image sets for validation. *Nat. Methods* 9, 637. <https://doi.org/10.1038/nmeth.2083>.
23. Maška, M., Ulman, V., Delgado-Rodriguez, P., Gómez-de-Mariscal, E., Nečasová, T., Guerrero Peña, F.A., Ren, T.I., Meyerowitz, E.M., Scherr, T., Löffler, K., et al. (2023). The Cell Tracking Challenge: 10 years of objective benchmarking. *Nat. Methods* 20, 1010–1020. <https://doi.org/10.1038/s41592-023-01879-y>.
24. Ma, J., He, Y., Li, F., Han, L., You, C., and Wang, B. (2024). Segment anything in medical images. *Nat. Commun.* 15, 654. <https://doi.org/10.1038/s41467-024-44824-z>.
25. Archit, A., Freckmann, L., Nair, S., Khalid, N., Hilt, P., Rajashekar, V., Freitag, M., Teuber, C., Spitzner, M., Tapia Contreras, C., et al. (2025). Segment Anything for Microscopy. *Nat. Methods* 22, 579–591. <https://doi.org/10.1038/s41592-024-02580-4>.
26. Chiu, C.-L., and Clack, N.; the napari community (2022). napari: a Python Multi-Dimensional Image Viewer Platform for the Research Community. *Microsc. Microanal.* 28, 1576–1577. <https://doi.org/10.1017/S1431927622006328>.